7. Les tests expliqués (`tests/`)

Nutri-Arbitre — fiche de révision · les tests vérifient chaque étape du trajet (clean → tools → garde_fou → agent → evaluation) · **R2** et preuves des briques 1, 3, 6, 8, 9

En une phrase : un test est une petite fonction qui lance une partie du code sur un exemple dont on connaît **déjà** la bonne réponse, calculée à la main, et qui vérifie que le code trouve la même. **42 tests**, répartis en 5 fichiers. Ils tournent en moins d'une seconde, **sans appeler Qwen** : ils donnent toujours le même résultat.
Lancer : `python -m pytest` (tous) ou `python -m pytest tests/test_garde_fou.py -v` (un seul fichier, avec le nom de chaque test).

Comment marche un test

```
def test_decrire_sucres_bio(df):
    r = tools.decrire(df, "sucres", bio=True)
    assert r["n"] == 2 and r["moyenne"] == 15.0      # (10 + 20) / 2
```

- **pytest** est l'outil qui lance les tests. Il cherche dans le dossier `tests/` les fichiers qui commencent par `test_`, et dans ces fichiers les fonctions qui commencent par `test_`. Il les lance toutes, une par une.
- `assert` veut dire « je vérifie que ». Si la condition est vraie, rien ne se passe ; si elle est fausse, le test **échoue** et pytest affiche la ligne et les valeurs.
- La bonne réponse est écrite **dans le test**, avec le calcul en commentaire : (10 + 20) / 2 = 15. On peut la vérifier à la main, sans faire confiance au code.

Les données de test : `conftest.py`

On ne teste pas sur les 953 vraies céréales : on ne pourrait pas calculer les réponses à la main. On invente **7 petits produits**, chacun avec un défaut choisi exprès :

Produit	Ce qu'il a de particulier	Ce que le nettoyage doit faire
1	bio (label `en:organic`), sans énergie	la garder, calculer l'énergie par INCO : 341 kcal
2	bio (label AB), « BIO VILLAGE » en majuscules, sodium sans sel	marque « bio village », sel = 0,2 × 2,5 = 0,5
3	Kellogg's	le garder
3 bis	même code-barres que le 3	retirer le doublon
4	sans nom	le retirer
5	sucres (90) > glucides (50), 150 g de sel, Nutri-Score « unknown »	mettre ces 3 valeurs à vide, garder le produit

6	aucune valeur nutritionnelle	le retirer

Résultat attendu : 7 – 1 doublon – 1 sans nom – 1 vide = **4 produits**, dont 2 bio.

```
12 @pytest.fixture
13 def produits_bruts():
33 @pytest.fixture
34 def df(produits_bruts):
35     return nettoyer(produits_bruts)[0]
```

Une **fixture** (`@pytest.fixture`) est une donnée préparée pour les tests. Un test qui écrit `df` entre ses parenthèses la reçoit automatiquement : pytest appelle la fonction `df()` et lui donne le résultat. Le fichier s'appelle `conftest.py` : pytest le lit tout seul, et ses fixtures servent à tous les fichiers de tests.

Ce que prouve chaque fichier

Fichier	Tests	Ce qu'il prouve	Brique
`test_clean.py`	9	le nettoyage retire les bonnes lignes, applique les règles INCO, vide les valeurs impossibles, retire les fiches incohérentes, regroupe les marques	6
`test_tools.py`	12	les outils donnent les moyennes, les marges et `difference_nette` calculées à la main	1, 8
`test_garde_fou.py`	15	on **essaie d'enfreindre** les deux règles, et le code bloque	3
`test_evaluation.py`	6	la façon de juger les réponses (juste, fausse, bloquée) et le calcul de la moyenne et de l'écart type	9

Quatre tests à savoir expliquer

1. La règle INCO (`test_clean.py` , lignes 20 à 22)

```
20 def test_regle_inco_energie_calculee(df):
21     # produit 1 : 4*60 + 4*10 + 9*5 + 2*8 = 341 kcal
22     assert df.loc[df["code"] == "1", "energie_kcal"].item() == 341
```

Le produit 1 n'a pas d'énergie. Le nettoyage doit la calculer : 4 × glucides + 4 × protéines + 9 × lipides + 2 × fibres. `df.loc[df["code"] == "1", "energie_kcal"]` prend la case « énergie » de la ligne dont le code est « 1 » (le masque), et `.item()` sort la valeur de cette case.

2. La marge d'erreur (`test_tools.py` , lignes 46 à 50)

```
47     # bio : 10 et 20 -> écart-type 7,07 ; erreur type 7,07 / racine(2) = 5 ; marge
1,96 x 5 = 9,8
48     r = tools.decrire(df, "sucres", bio=True)
49     assert r["marge_95"] == 9.8
50     assert r["intervalle_95"] == [5.2, 24.8]             # 15 - 9,8 et 15 + 9,8
```

Toute la brique 8 tient dans le commentaire : on refait le calcul à la main et on compare.

3. Une erreur attendue (`test_tools.py`, lignes 30 à 32)

```
30 def test_nutriment_inconnu_refuse(df):
31     with pytest.raises(ValueError):
32         tools.decrire(df, "vitamine_c")
```

Ici, le test **réussit si le code lève une erreur**. `with pytest.raises(ValueError):` veut dire « la ligne en dessous doit provoquer une ValueError ». Si elle ne la provoque pas, le test échoue : l'outil aurait accepté un nutriment qui n'existe pas.

4. Essayer d'enfreindre la règle (`test_garde_fou.py`, lignes 36 à 42)

```
39     resultat = [{"nutriment": "sucres", "n": 543, "moyenne": 13.17, "marge_95":
0.69}]
40     question = "Combien de grammes de sucre y a-t-il dans un bol de 40 g de
céréales bio ?"
41     reponse = "100 g contiennent 13,2 ± 0,7 g de sucre. Pour un bol de 40 g,
environ 5,3 g."
42     assert verifier(reponse, resultat, question) == (False, ["5,3"])
```

C'est la vraie réponse de Qwen du 2 novembre, recopiée dans le test. On **simule** le modèle : pas besoin de le lancer, on lui fait « dire » une réponse qui contient un chiffre non calculé. Le test vérifie que le garde-fou refuse, et qu'il désigne exactement le bon nombre (5,3). 13,2, 0,7 et 40 sont acceptés : ils viennent de l'outil ou de la question.

Pourquoi les tests n'appellent jamais Qwen

- **Qwen ne répond pas deux fois pareil** : un test qui l'appellerait passerait une fois et échouerait la suivante. Un test doit toujours donner le même résultat.
- **Qwen est lent** : 30 s par question. Les 42 tests tournent en moins d'une seconde.
- Le comportement réel de Qwen est mesuré ailleurs : dans **la brique 9** (`evaluation.py`, 3 lancements par question) et dans les traces.

Les limites (à dire soi-même)

- Un test ne prouve que **les cas qu'on a pensé à écrire**. Le filtre bio non demandé n'était testé nulle part, jusqu'à ce qu'on le voie dans une vraie trace.
- Les données de test sont inventées : elles ne garantissent pas que les 953 vraies fiches n'ont pas un défaut auquel on n'a pas pensé.
- `download.py` n'a pas de test : il dépend d'Internet et de l'API d'Open Food Facts.

Questions d'examinateur

> 1. Qu'est-ce qu'un test, et que fait `assert` ?
> 2. Comment connaissez-vous la bonne réponse d'un test ?
> 3. Pourquoi ne pas tester sur les 953 vraies céréales ?
> 4. Qu'est-ce qu'une fixture ? Comment un test reçoit-il `df` ?
> 5. Pourquoi 7 produits au départ et 4 à l'arrivée ?
> 6. Que vérifie `with pytest.raises(ValueError):` ?
> 7. Comment testez-vous le garde-fou sans lancer Qwen ?
> 8. Pourquoi aucun test n'appelle-t-il le modèle ?
> 9. Vos tests passent tous. Le système est-il donc sans erreur ?

Corrigé

1. Une petite fonction qui lance une partie du code sur un exemple à réponse connue. `assert` vérifie une condition : si elle est fausse, le test échoue et pytest montre où.
2. Je la calcule **à la main**, et le calcul est écrit en commentaire dans le test. Exemple : la marge 1,96 × 7,07 / √2 = 9,8.
3. On ne pourrait pas calculer les réponses à la main, et un défaut précis (doublon, sel impossible) ne serait pas forcément présent. Les petits produits inventés ont chacun un défaut choisi exprès.
4. Une donnée préparée pour les tests, avec `@pytest.fixture`, dans `conftest.py`. Un test qui écrit `df` en paramètre la reçoit : pytest appelle la fixture `df`, qui nettoie les 7 produits.
5. On retire le doublon (même code-barres), le produit sans nom et le produit sans aucune valeur. Le produit 5, qui a des valeurs impossibles, est gardé : seules ces cases sont mises à vide.
6. Que la ligne en dessous **provoque** une ValueError. Le test réussit si le code refuse, par exemple un nutriment « vitamine_c » qui n'existe pas.
7. On **simule** sa réponse : on écrit dans le test une réponse avec un chiffre non calculé (la vraie réponse « 5,3 g » du 2 novembre) et on vérifie que `verifier` la refuse.
8. Qwen ne répond pas deux fois pareil et met 30 s par question : un test doit être rapide et donner toujours le même résultat. Le vrai comportement du modèle est mesuré par la brique 9.
9. **Non.** Les tests vérifient les cas qu'on a pensé à écrire. Les traces réelles et la brique 9 ont montré des erreurs qu'aucun test ne couvrait (le filtre bio non demandé) : on a alors ajouté la règle et le test.